

DYAD FINANCE

**Arquitectura Basada en Diseno
Orientado por Dominio (DDD)**

Rol: Software Architect & Web3 Developer

Autor: Antigravity AI Partner

Fecha: Junio 2026

Version: 2.0 (Produccion)

1. Introduccion a DDD en Dyad Finance

El protocolo Dyad Finance combina logica financiera de estabilidad cripto (L1) con plusvalia inmobiliaria tokenizada en activos reales (RWA). A medida que el sistema escala y se integran multiples integraciones de oraculos, puentes de liquidez y dinamicas de Staking, la arquitectura tradicional de frontend acoplada empieza a presentar problemas de mantenibilidad, fallas de testing y riesgo de inconsistencias.

La adopcion de Domain-Driven Design (DDD) resuelve estos desafios separando claramente las reglas de negocio del protocolo (el Dominio) de la implementacion tecnica (MetaMask, contratos inteligentes, interfaces graficas y red).

Beneficios Clave:

- Independencia Tecnologica: Las reglas matematicas de LTV, emision y plusvalia se definen de forma pura, facilitando las pruebas unitarias sin mockear Web3.
- Bounded Contexts Claros: Evita que el contexto de Stablecoin se contamine con el contexto de construccion fisica o con el calculo del airdrop de puntos.

2. Bounded Contexts (Contextos Acotados)

A. Contexto de Estabilidad (Vault Context)

Responsable de las operaciones principales de colateral y deuda de la stablecoin. Maneja el ratio de salud (LTV) y la acuñacion del token estable usdJ y volatil jETH.

- Aggregate Root: Vault
- Entities: CollateralPosition, DebtAccount
- Value Objects: LtvRatio, AssetPrice, StableAmount

B. Contexto de Activos Reales (RWA Context)

Maneja la representacion digital de bienes raices on-chain. Gestiona el registro de propiedades, carga de fotografias de evidencia de avance de construccion, tasaciones de peritos oficiales y firmas criptograficas del oraculo.

- Aggregate Root: RealEstateProperty
- Entities: RwaCertificate (NFT ERC721), TasacionProposal
- Value Objects: PropertyValue, ConstructionStage, OracleSignature

C. Contexto de Staking & Minería (Yield Context)

Encargado de los incentivos liquidos. Coordina el bloqueo del RwaCertificate NFT, el calculo matematico en tiempo real de recompensas basándose en la tasa base, el valor tasado y los multiplicadores dinamicos de avance de obra (Gris, 3D, Terminada).

- Aggregate Root: StakingPool
- Entities: Staker, MiningPosition
- Value Objects: EmissionRate, RewardMultiplier, AccumulatedYield

3. Arquitectura de Capas (Arquitectura Cebolla)

Para mantener el dominio aislado de librerias Web3 como Ethers.js, proponemos una arquitectura Onion estructurada en cuatro capas independientes:

1. Capa de Dominio (Domain): Es la mas interna. Contiene las definiciones de entidades, aggregates y reglas matematicas puras. No puede importar nada ajeno a JS basico. Define interfaces (puertos) para repositorios externos.
2. Capa de Aplicacion (Application): Define los casos de uso (ej. depositar colateral, reclamar tokens minados). Actua como orquestador de la logica ejecutando comandos y emitiendo eventos de dominio.
3. Capa de Infraestructura (Infrastructure): Implementa la comunicacion tecnica. Aqui viven los adaptadores de smart contracts (Ethers.js), firmas de transacciones, llamadas RPC a Sepolia e interacciones con el localStorage.
4. Capa de Interfaz (UI): Next.js 14/15, componentes React y hooks visuales. Su unico proposito es pintar el estado del sistema y enviar intenciones de usuario a la capa de aplicacion.

4. Propuesta de Estructura de Directorios (DDD)

```
src/
-- domain/                # Logica y Reglas de Negocio Puras
-- shared/                # Entidades Compartidas y Value Objects
-- vault/                 # Bounded Context de la Boveda
--   entities/            # CollateralPosition, DebtAccount
--   value-objects/       # LtvRatio.ts, StableAmount.ts
--   repository/          # IVaultRepository.ts (Interfaces)
--   events/              # CollateralDeposited.ts
-- rwa/                   # Bounded Context de Bienes Raices RWA
-- yield/                 # Bounded Context de Staking & Minería
-- application/           # Casos de Uso del Protocolo
--   vault/               # Casos de uso de Boveda
--     commands/          # DepositCollateralUseCase.ts
--     queries/           # GetAccountHealthQuery.ts
--     yield/             # ClaimMiningRewardsUseCase.ts
-- infrastructure/        # Adaptadores y Detalles Tecnicos
--   blockchain/          # Interacciones con Ethers.js / Sepolia
--     ethers-vault-repository.ts
--     contract-abis/
-- storage/               # Cache, LocalStorage adapters
-- ui/                    # Interfaz de Usuario y Componentes
--   components/          # ActionCards.tsx, CanvasScrollytelling.tsx
--   hooks/               # useVaultDomain.ts (React Hooks de integracion)
--   pages/               # Enrutamiento de Next.js
```

5. Conclusion y Siguietes Pasos

La implementacion de esta arquitectura en Dyad Finance desacoplara de forma definitiva la UI del estado asincrono de la blockchain de Ethereum. Permitira simular y testear localmente el motor de minería de RWA y el calculo de LTV sin necesidad de realizar llamadas RPC de red constantes, protegiendo al frontend de fallas externas de conexion.

Para proceder, se recomienda comenzar por extraer las formulas matematicas de LTV y Minería en modulos independientes de dominio (`src/domain/`) y agregar las pruebas unitarias correspondientes con Vitest / Jest.